

Bheeshm Phase 2 Proposal - Runtime Profiles

Bheeshm Phase 2 Proposal

Runtime Profiles for ACSIL Studies

Target environment: Quarkus + HTML frontend + SQLite3 backend

Document date: 2026-03-15

1. Objective

Phase 2 extends Bheeshm from a calibration JSON publisher into a controlled runtime profile publisher for ACSIL studies.

The goal is to support stable, study-specific, instrument-specific, chart-specific, intent-specific parameter sets without exposing those values directly in Sierra Chart study inputs.

Core outcome: Bheeshm stores and publishes one approved runtime JSON file per active profile, and the ACSIL study loads that file locally at runtime.

2. Problem Statement

Current study parameters are often embedded directly in ACSIL inputs or code. That creates four problems:

- Proprietary parameter sets are exposed to end users.
- Large studies become difficult to manage across multiple instruments, chart types, and target intents.
- The same study may need multiple profiles for the same instrument.
- Promoting a new live parameter set is operationally harder than replacing a single published file.

This is especially relevant for large custom studies such as `K_Util_ModeBaton_V2.cpp`, where profiles can differ by instrument, chart type, session, and target intent while remaining otherwise stable for long periods.

3. Design Principles

- Keep ACSIL consumption local and file-based. Studies should not query Bheeshm directly.
- Use one JSON file per live profile, not one giant file for all studies.
- Share only minimal metadata conventions. Payload parsing remains study-specific.
- Separate calibration data from reference/runtime profile data.
- Treat profile publication as a lifecycle with draft, approved, and live states.

4. Scope of Phase 2

In scope

- Metadata model for runtime profiles.
- SQLite schema for profile storage and promotion.

- JSON envelope standard for published profile files.
- Quarkus service responsibilities for CRUD, validation, publish, and lifecycle transitions.
- Frontend screens for listing, editing, promoting, and publishing profiles.

Out of scope

- A generic schema that fully describes every custom study payload.
- Direct Sierra-to-server communication.
- Full client-side secrecy if the JSON file itself is delivered to the client machine.

5. Conceptual Model

A runtime profile is identified by the business meaning of the settings, not by the physical file path.

Field	Description
study_name	The consuming study family, for example MODE_BATON.
instrument	The instrument symbol or canonical instrument key, for example MES.
chart_type	The chart context label, for example 1MIN or 30T.
session_type	Session scope such as ALL, RTH, or GLOBEX.
target_intent	The intended use or target purpose, for example ORB or REVERSAL.
version	Version number of the profile payload.

This supports multiple profiles for the same study and same instrument as long as they differ by chart type, session type, target intent, or version.

6. Data Separation

Calibration data and runtime profiles should be stored separately.

7. SQLite Schema Proposal

7.1 Primary table: runtime_profiles

```
CREATE TABLE runtime_profiles (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  study_name TEXT NOT NULL,
  instrument TEXT NOT NULL,
  chart_type TEXT NOT NULL,
```

```

session_type TEXT NOT NULL,
target_intent TEXT NOT NULL,
version INTEGER NOT NULL,
config_type TEXT NOT NULL DEFAULT 'reference',
status TEXT NOT NULL DEFAULT 'draft',
schema_version TEXT NOT NULL,
title TEXT,
description TEXT,
payload_json TEXT NOT NULL,
payload_hash TEXT,
effective_from_utc TEXT,
created_at_utc TEXT NOT NULL,
updated_at_utc TEXT NOT NULL,
created_by TEXT,
updated_by TEXT,
approved_at_utc TEXT,
approved_by TEXT,
published_at_utc TEXT,
published_by TEXT,
published_file_path TEXT,
notes TEXT,
CHECK (status IN ('draft', 'approved', 'live', 'archived')),
CHECK (config_type IN ('reference', 'calibration'))
);

```

7.2 Uniqueness constraints

```

CREATE UNIQUE INDEX ux_runtime_profiles_identity_version
ON runtime_profiles (
  study_name,
  instrument,
  chart_type,
  session_type,
  target_intent,
  version
);

```

```

CREATE UNIQUE INDEX ux_runtime_profiles_one_live
ON runtime_profiles (
  study_name,
  instrument,
  chart_type,
  session_type,
  target_intent
)
WHERE status = 'live';

```

7.3 Query support indexes

```

CREATE INDEX ix_runtime_profiles_lookup
ON runtime_profiles (study_name, instrument, chart_type, session_type, target_intent, status);

CREATE INDEX ix_runtime_profiles_updated

```

ON runtime_profiles (updated_at_utc DESC);

7.4 Optional audit table: runtime_profile_events

```
CREATE TABLE runtime_profile_events (  
id INTEGER PRIMARY KEY AUTOINCREMENT,  
runtime_profile_id INTEGER NOT NULL,  
event_type TEXT NOT NULL,  
event_at_utc TEXT NOT NULL,  
actor TEXT,  
details_json TEXT,  
FOREIGN KEY(runtime_profile_id) REFERENCES runtime_profiles(id)  
);
```

SQLite note: storing payload_json as TEXT is sufficient. Quarkus can validate and manipulate JSON at the application layer using Jackson or JSON-B, while SQLite remains simple and reliable.

8. Runtime JSON Envelope

The published file should use a stable top-level envelope and a study-specific parameters payload.

```
{  
"schema_version": "1.0",  
"config_type": "reference",  
"study_name": "MODE_BATON",  
"instrument": "MES",  
"chart_type": "1MIN",  
"session_type": "RTH",  
"target_intent": "ORB",  
"version": 3,  
"status": "live",  
"generated_at": "2026-03-15T14:30:00Z",  
"generated_by": "bheeshm",  
"title": "MES 1MIN ORB RTH",  
"description": "Live runtime profile for Mode Baton on MES 1MIN ORB RTH.",  
"parameters": {  
"example_field_a": 1.25,  
"example_field_b": 8,  
"example_field_c": true  
}  
}
```

8.1 Envelope fields

Field	Required	Description
-------	----------	-------------

schema_version	Yes	Version of the published runtime envelope.
----------------	-----	--

config_type	Yes	Usually reference for Phase 2 profiles.
-------------	-----	---

study_name	Yes	Consuming study family.
------------	-----	-------------------------

| instrument | Yes | Instrument key.

| chart_type | Yes | Chart context label such as 1MIN or 30T.

| session_type | Yes | ALL, GLOBEX, or RTH.

| target_intent | Yes | Target use such as ORB, REVERSAL, or another controlled intent key.

| version | Yes | Integer version of this profile instance.

| status | Yes | Normally live in published artifacts.

| generated_at | Yes | UTC timestamp of file generation.

| generated_by | No | Publisher name or user.

| title | No | Human-friendly label.

| description | No | Human-friendly description.

| parameters | Yes | Study-specific payload consumed by custom ACSIL loader logic.

9. File Publication Layout

Bheeshm should publish one file per live profile under a deterministic folder structure.

```
live/  
mode_baton/  
mes/  
mes_1min_rth_orb_v3.json  
mes_30t_globex_reversal_v1.json  
nq/  
nq_1min_rth_orb_v2.json
```

```
incoming/  
mode_baton/  
mes/  
mes_1min_rth_orb_v4_candidate.json
```

Recommended file naming pattern:

<instrument>_<chart_type>_<session_type>_<target_intent>_v<version>.json

The physical file path is operational metadata. The file contents remain the source of truth for validation inside the study.

10. Quarkus Backend Responsibilities

- Create and edit runtime profiles.
- Validate mandatory identity fields and payload presence.
- Track profile versions and statuses.
- Promote one version to live while demoting the prior live version for the same logical

profile.

- Publish the generated JSON artifact to the configured folder.
- Record audit events for approval and publication.

10.1 Suggested service split

| Service | Responsibility

| RuntimeProfileService | CRUD, identity validation, version handling.

| RuntimeProfilePublishService | Generate final JSON envelope and write files to the publish target.

| RuntimeProfileAuditService | Write lifecycle events such as approve, publish, archive.

| RuntimeProfileQueryService | Search and filter profiles for UI listing pages.

10.2 Suggested REST endpoints

```
GET /api/runtime-profiles
GET /api/runtime-profiles/{id}
POST /api/runtime-profiles
PUT /api/runtime-profiles/{id}
POST /api/runtime-profiles/{id}/approve
POST /api/runtime-profiles/{id}/publish
POST /api/runtime-profiles/{id}/archive
GET /api/runtime-profiles/{id}/events
```

11. HTML Frontend Responsibilities

- List profiles by study, instrument, algo, session, and status.
- Create and edit profile metadata and payload JSON.
- Show version history.
- Approve and publish profiles explicitly.
- Display the final published path and publication timestamp.

11.1 Minimal screens

- Profile list screen
- Profile edit/detail screen
- Version/event history panel
- Publish confirmation dialog

12. ACSIL Consumption Pattern

The ACSIL side should remain simple. Each study loads one file, validates the shared envelope fields, binds the parameters payload into a study-specific typed struct, caches the result, and then uses plain variables in the study body.

Important: Do not force a generic universal payload model across all studies. Each study should keep its own typed runtime config struct and loader logic for its own fields.

Recommended split on the ACSIL side:

- Minimal shared envelope and JSON helper utilities.
- Study-specific payload structs and loader functions.
- No repeated JSON inspection scattered across the main study logic.

13. Example Lifecycle

- User creates or edits profile `MODE_BATON / MES / 1MIN / RTH / ORB / v3`.
- Bheeshm stores it as version 3 with status draft.
- User reviews and approves it.
- User publishes it to live.
- Bheeshm writes the final JSON file to the configured live folder.
- The Sierra study reads that file locally and validates the envelope.

14. Risks and Constraints

- If the client can access the JSON file on disk, the values are still readable. This is operational hiding, not strong secrecy.
- Payload shapes will vary significantly by study. That is expected and should not be over-abstracted.
- The live folder path and write permissions need to be explicitly configured in Quarkus.
- Version and status transitions should be guarded to prevent multiple live versions of the same logical profile.

15. Recommended First Implementation

- Add the `runtime_profiles` and optional `runtime_profile_events` tables.
- Implement basic Quarkus CRUD and publish endpoints.
- Create one HTML workflow for managing runtime profiles.
- Use one large study, likely Mode Baton, as the first Phase 2 consumer.
- Keep the current calibration flow unchanged while proving the new profile flow separately.

16. Final Recommendation

Bheeshm Phase 2 should store runtime profiles in separate SQLite tables, publish them as one JSON file per live profile, and keep ACSIL consumption local, typed, and study-specific. This gives operational control, versioning, cleaner deployment, and reduced exposure of sensitive settings in Sierra study inputs without overbuilding a universal schema framework.